

Running the site locally

Classic and International Pro editions — a step-by-step setup guide

Version 1.0.0

This guide covers two separate things: running the ready-made ZIP downloads from the GitHub Release, and running the whole project from source. Start with Part 1 if you only want to look at the websites. Use Part 2 if you want live reloading, the staff dashboard with a real API, or you plan to change the code.

The two editions

Edition	What it is	Where it can be served from
Classic	The original main-site layout.	The root of a site only.
International Pro	Editorial, responsive experience with destination search, journey filters, shortlists and package planning.	Any folder, including a sub-path.

Both editions share the same tour and transport model, a persistent Bangla / EN language switch, teal, sky-blue and orange branding, and locally hosted Bangla and English fonts. No internet connection is needed once a build is downloaded.

Before you start

Part 1 needs nothing but a web browser and a way to start a tiny local web server (Python and Node both work, and one of them is almost certainly already on your machine). Part 2 needs Node.js 22 and npm.

Part 1 — Run the downloaded ZIPs

Download either or both archives from the Releases page of the repository, then unzip them. The Classic archive unpacks into a folder named "classic"; the Pro archive unpacks into "pro".

Do not double-click index.html

Opening index.html straight from your file manager uses a file:// address, and the page will stay blank. Browsers refuse to load JavaScript modules over file://, which is how both editions are built. You must serve the folder over http:// using one of the small servers below. This is a browser security rule, not a fault in the build.

Option A — Python (already installed on macOS and most Linux systems)

Open a terminal, move into the unzipped folder, and start the built-in server:

```
# International Pro
cd pro
python3 -m http.server 8080

# Classic — same command, different folder
cd classic
python3 -m http.server 8080
```

Then open `http://localhost:8080/` in your browser. Press `Ctrl+C` in the terminal to stop the server.

On Windows, use `"py -m http.server 8080"` if `"python3"` is not recognised.

Option B — Node.js

If you have Node.js installed, no separate install step is needed:

```
cd pro
npx serve -l 8080

# or
npx http-server -p 8080
```

`npx` will ask once to download the package, then print the address to open.

Which address to open

Edition	Serve this folder	Then open
Classic	<code>classic/</code>	<code>http://localhost:8080/</code>
International Pro	<code>pro/</code>	<code>http://localhost:8080/</code>

Classic must sit at the site root

The Classic bundle loads its photography from absolute paths such as `/media/sajek-1.webp`. If you serve a parent folder and browse to `/classic/`, the page will load but every photo will be missing. Always point the server at the classic folder itself, so that `index.html` is at the root of what is being served. The Pro edition has its imagery bundled into `assets/`, so it does not have this restriction.

Viewing both editions side by side

Run two servers on different ports — for example Classic on 8080 and Pro on 8081 — by opening a second terminal window and passing a different port number. Inside the Pro edition, the "Classic edition" link in the header is controlled by a single line in `pro/index.html`:

```
<meta name="classic-edition-url" content="../" />
```

Change the content value to wherever you put Classic, such as `http://localhost:8080/`, and reload the page.

Part 2 — Run the full project from source

This gives you live reloading, the staff dashboard, and a real shared API instead of browser-only demo storage. You need Node.js 22 and npm.

1. Get the code and install dependencies

```
git clone https://github.com/Salman-Sazid-007/Rajshahi_tour_-_travels.git
cd Rajshahi_tour_-_travels

npm ci --prefix backend
npm ci --prefix frontend
```

2. Static preview of both editions

Build both editions and serve the published output together:

```
npm run build:pages
npm run preview
```

The preview binds 0.0.0.0:4173. Its default page opens Pro at `/Rajshahi_tour_-_travels/pro/`, and Classic is at `/Rajshahi_tour_-_travels/`. The paths `/pro/` and `/docs/` serve the standalone copies.

3. Full stack with the real API

To use shared availability and operator-visible requests instead of demo storage:

```
npm run build
npm start
```

The Express server binds 0.0.0.0:3000 and serves Classic plus `/pro/` from one origin.

4. Development servers with live reload

Start the backend first, then run the edition you are working on in a second terminal:

```
# terminal 1 — API
npm run dev

# terminal 2 — Classic with live reload
npm --prefix frontend run dev

# terminal 2 (alternative) — International Pro
cd frontend && npx vite --config vite.pro.config.js
```

Development API calls use relative URLs through Vite's proxy, so the browser never needs to know the backend's address.

Ports at a glance

Port	Started by	Serves
8080	<code>python3 -m http.server</code> (Part 1)	A single unzipped edition
4173	<code>npm run preview</code>	Static preview of both editions
3000	<code>npm start</code> / <code>npm run dev</code>	Express API, Classic and <code>/pro/</code>
5173	<code>npm --prefix frontend run dev</code>	Classic dev server with live reload

Troubleshooting

Symptom	Cause and fix
Page is completely blank	You opened <code>index.html</code> as a <code>file://</code> address. Serve the folder over <code>http://</code> instead — see Part 1.
Photos missing in Classic	Classic is not being served at the site root. Point the server at the classic folder itself, not its parent.
Port already in use	Another program holds that port. Pass a different number, for example 8081.
"python3: command not found"	Use Option B with Node, or try <code>"py -m http.server 8080"</code> on Windows.
Bookings vanish after a refresh	Expected without a running API. The static builds fall back to browser-local demo storage. Use Part 2, step 3.
Another device cannot connect	Localhost only serves your own machine. Bind to 0.0.0.0 and use your machine's LAN address.

Important: demo data versus real sales

Nothing here takes real payments

All bKash handling in this project is manual. Entering a transaction reference does not charge money, verify a payment, or confirm a reservation. No payment gateway has been integrated, and cancelling a booking does not refund anything automatically.

- A static build with no API reachable uses clearly labelled browser-local demo storage. Bookings, seat maps and approvals made in your browser are not visible to anyone else.
- WhatsApp links open a prepared message; the customer still has to send it.
- Seeded tours, statistics, traveler stories and staff accounts are demonstration content. Fictional demo phone numbers start with 000.

- Pro's optional USD display uses a fixed demo conversion, not a live exchange rate. Final prices remain BDT.
- Before running a real backend, read the "Before operating a real backend" section of README.md. Prototype role shortcuts are not real authentication.

Rebuilding the archives yourself

The two ZIPs on the Releases page are produced from source by:

```
npm ci --prefix frontend
npm run package:editions
```

The archives are written to release-artifacts/. To regenerate this PDF:

```
npm install --no-save pdfkit
node scripts/make-local-guide.mjs
```

Verification commands, publishing paths and the full operational notes live in README.md at the root of the repository.